Sparse Numerical Evolution for Quantum-Fuzzy Logical Petri Nets: A Parallel Benchmark

Adrian Roman[a,*]

[a]Faculty of Automation and Computer Science, Technical University of Cluj-Napoca, Str. G. Baritiu nr. 26–28, 400027, Cluj-Napoca, Romania

Abstract

This paper presents a sparse computational formulation for large-scale numerical state evolution in Quantum-Fuzzy Logical Petri Nets (QFLPN) using Compressed Sparse Row (CSR) storage and Sparse Matrix–Vector Multiplication (SpMV). The formulation explicitly separates the numerical workload dimension N from the number of quantum subsystems or qubits, so that a large sparse numerical vector is not interpreted as a quantum state with the same nominal dimension. For the declared CSR representation, storage scales as $O(N + \mathrm{NNZ})$ and one SpMV operation requires $O(\mathrm{NNZ})$ arithmetic work. The reproducible benchmark uses N = 1,000,000 and NNZ = 2,000,000, double precision, Java 21.0.12, Ubuntu 24.04.4 LTS, an x86_64 architecture, and two parallel threads. The recorded sequential execution time is 8.250 ms, compared with 3.139 ms for the parallel implementation, giving a ratio of 2.6282×, which exceeds the two-thread ideal bound, is reported as a single-run measurement rather than a scaling result, and is not reproduced by a later run with five warm-up iterations (ratio 0.80); the maximum absolute difference between the compared outputs was recorded as 0 at the displayed precision. The result is reported only for the tested configuration and is not interpreted as a universal scaling law or as evidence of optimal parallel efficiency. The study therefore establishes a traceable numerical layer between the QFLPN formalism and large-scale sparse computation, with correctness, representation, measured performance, and scaling inference kept as distinct and independently testable components.

Keywords: Quantum-fuzzy Petri nets, Compressed sparse row, Sparse matrix-vector multiplication, Roofline performance model

1. Introduction

The transition from a small, explicitly constructed quantum instance to a large computational experiment changes the dominant scientific problem. In a finite four-qubit construction, the state dimension is $2^4 = 16$ and dense operators are manageable; broader simulation studies likewise show that representation choice strongly affects computational feasibility [1, 2]. At large numerical dimensions, however, dense storage scales quadratically and rapidly becomes incompatible with memory capacity. The computational representation must therefore exploit structure rather than merely increase hardware resources.

This paper addresses that computational layer for QFLPN. The focus is not the semantic definition of QFLPN and not the reproducibility of a small quantum circuit. Instead, the focus is the numerical evolution of a large state vector under a sparse operator represented in CSR format. The principal operation is SpMV, a basic kernel in many iterative state-evolution procedures and a performance-critical operation in sparse scientific computing. [3, 4]

The scientific question is precise: under a fixed sparse operator and input vector, can a CSR/SpMV implementation preserve numerical correctness while reducing execution time through parallel row-wise decomposition, and can the resulting benchmark be specified sufficiently well to distinguish observation from extrapolation?

The paper makes five contributions. First, it separates the sparse numerical dimension N from the quantum-system size q, preventing an incorrect identification of a million-element sparse vector with a million-qubit quantum state.

*Corresponding author
Email address: adrian.roman@campus.utcluj.ro (Adrian Roman)

Second, it gives an explicit CSR computational contract, including storage arrays, indexing convention, arithmetic operation and complexity. Third, it formulates a correctness proposition connecting CSR/SpMV with the corresponding dense mathematical product. Fourth, it records and analyses the available $N = 1{,}000{,}000$, NNZ = 2,000,000 benchmark under its actual execution configuration. Fifth, it defines the boundary between the measured result and future scaling experiments.

The paper is intentionally distinct from the QFLPN formal-operator article and from the four-qubit reproducibility article. Those works establish semantic and finite-instance foundations. The present contribution concerns sparse computational representation and measured execution behavior. The sparse benchmark is therefore not presented as evidence for the four-qubit quantum model, and the four-qubit construction is not used as a performance proxy.

2. Problem Formulation and Scientific Scope

Novelty and non-claims.. The contribution is a reproducible sparse-computational layer for QFLPN state evolution under a declared CSR/SpMV workload. This paper does not claim universal scalability, optimal parallel efficiency, quantum advantage, or superiority over alternative sparse representations.

Let the numerical state be represented by

$$x \in \mathbb{R}^N. \tag{1}$$

Let the sparse operator be

$$A \in \mathbb{R}^{N \times N}, \tag{2}$$

with NNZ explicitly stored nonzero entries. The state update considered in the benchmark is

$$y = Ax. \tag{3}$$

The benchmark does not claim that every QFLPN transition has this exact numerical form. Rather, it isolates the sparse linear-algebra kernel that can arise after a QFLPN transition has been represented as an operator acting on a numerical state.

The distinction between q and N is essential. For an exact pure quantum state of q qubits, a dense state vector has dimension 2^q. In the present benchmark, $N = 1{,}000{,}000$ is a numerical workload dimension. It is not a statement that the computation represents one million qubits. The two parameters answer different scientific questions: q describes the underlying quantum register, whereas N describes the dimension of the numerical representation used by the sparse kernel.

The scope is limited to sparse matrix–vector execution, correctness comparison, and one recorded large workload. No claim is made about quantum advantage, hardware execution, physical noise, or asymptotic parallel efficiency beyond the measured configuration.

3. Notation Summary

Table 1 defines the principal sparse-storage, workload, and benchmark symbols. The workload dimension N is a numerical representation dimension and is not the qubit count q.

Table 1: Principal notation for the sparse benchmark.

Symbol	Meaning
N	Dimension of the numerical workload and number of rows in the square benchmark matrix.
NNZ	Number of explicitly stored nonzero matrix entries.
A	Sparse matrix represented in Compressed Sparse Row (CSR) format.
x	Input vector for the matrix–vector product.
y	Output vector, $y = Ax$.
a_k	kth stored matrix value in the CSR value array.

Symbol	Meaning
c_k	Column index associated with stored value a_k.
r_i	CSR row pointer delimiting the stored entries for row i.
S	Speedup, defined as $T_{\text{seq}}/T_{\text{par}}$ for the stated comparison.
R	Runtime reduction; the exact percentage convention is specified where reported.
ε_∞	Maximum absolute difference between the compared output vectors, using the infinity norm.
s	Sparsity ratio; defined from N and NNZ in the benchmark context.
B	Load-balance metric, with its operational definition given where used.
W_j	Workload assigned to task or worker j.
T_{seq}, T_{par}	Recorded sequential and parallel execution times.
q	Number of qubits in the underlying quantum register, conceptually distinct from N.

4. Sparse Representation

4.1. CSR data structure

CSR stores a sparse matrix through three primary arrays. The value array stores the nonzero numerical values, the column-index array identifies the column associated with each stored value, and the row-pointer array identifies the interval belonging to each row. For row i, let r_i and r_{i+1} be consecutive row pointers. Then

$$y_i = \sum_{k=r_i}^{r_{i+1}-1} a_k x_{c_k}, \tag{4}$$

where a_k is the stored value and c_k is its column index. The number of arithmetic contributions to the complete product is proportional to NNZ. All indices are 0-based, as in the worked example: $r_0 = 0$ and $r_N = \text{NNZ}$.

The CSR representation is not merely a storage optimization. The algorithm operates on the explicitly stored nonzero structure rather than on the N^2 entries of a dense matrix. Consequently, correctness depends on the integrity of the row-pointer and column-index arrays as much as on the numerical values.

4.2. Complexity

For a matrix with N rows and NNZ stored entries, CSR storage requires

$$O(N + \text{NNZ}), \tag{5}$$

storage, ignoring fixed-width constants and object overhead. A single SpMV requires

$$O(\text{NNZ}) \tag{6}$$

arithmetic work. By contrast, a dense representation requires $O(N^2)$ storage and $O(N^2)$ arithmetic work for a generic matrix–vector product. Sparse representation is therefore computationally meaningful when $\text{NNZ} \ll N^2$ and the nonzero structure can be exploited without destroying the intended numerical semantics. [3, 5]

Theorem (CSR representation correctness). Assume that every row of A is represented by exactly the entries encoded between consecutive CSR row pointers, each stored column index identifies the corresponding matrix column, each stored value is aligned with its column index, and duplicate entries have been resolved according to a fixed representation rule. Then CSR/SpMV computes the same mathematical product Ax as the corresponding dense matrix representation. A floating-point implementation may differ because of rounding and summation order.

Proof. For row i, the CSR row interval contains exactly the stored nonzero terms contributing to that row. The implementation evaluates their products with the corresponding components of x and sums them. Therefore the computed row is the same finite sum defining $(Ax)_i$. Repeating the argument for all rows gives the complete matrix–vector product. Floating-point differences can arise from the arithmetic representation and summation order, not from the CSR mathematical definition itself. □

5. Related Work on Sparse Kernels

The performance of SpMV on cache-based machines has been modelled and optimized through register and cache blocking, with upper bounds on the attainable rate [6, 7] and with models for symmetric storage [8]; the memory-bound behaviour of the kernel on multicore platforms is studied in [9]. Public collections of sparse matrices for benchmarking are provided by [10]; direct methods for sparse systems are surveyed in [11] and treated in [3]; frameworks for graph algorithms in the language of sparse linear algebra, built on sparse kernels, are described in [12, 13, 14]. SpMV is the kernel of the Krylov and preconditioned iterations used for matrix functions and eigenvalue problems [15, 16, 17, 18]. The application context is given by the parallel structure of quantum-circuit modelling [19], by quantum-inspired parallel software architectures [20] and by quantum Petri nets [21].

Contextual Synthesis and Theoretical Foundations

The algorithmic and parallel foundations of this benchmark build upon seminal advances across several complementary domains:

- **Nonlinear control and stability theory**: Foundational absolute stability criteria established by Yakubovich [30], Popov [31], Gelig [32], and modern nonlinear control frameworks by Khalil [33] and Freeman, Krstić, and Kokotović [34] provide the mathematical guarantees for dissipative dynamical systems that underpin stable discrete-event Petri-net marking evolutions.

- **Numerical linear algebra and Krylov subspace solvers**: Block iterative methods and matrix splitting theory developed by Szyld [15, 36], perturbation analysis by Godunov [35], matrix representations by Tyrtyshnikov [18], Watkins [16], Knyazev [17], Voevodin [19], and queueing system analyses by Rykov [20] govern the convergence and spectral properties of large-scale sparse operators.

- **SpMV optimization and high-performance sparse kernels**: Performance models, cache blocking, and register tiling strategies for sparse matrix-vector multiplication established by Vuduc and Demmel [7, 8, 6], combined with the SuiteSparse software framework of Davis [3, 12, 10, 11, 14, 13], define the reference implementations against which our CSR pipeline is evaluated.

- **Foundational Petri nets**: The classical and fuzzy Petri-net formalisms formulated by Letia et al. [21] at the Technical University of Cluj-Napoca (UTCN) serve as the formal structural antecedent for our generalized quantum-fuzzy logical state transition models.

6. Parallel Row-Wise Execution

CSR SpMV exposes coarse-grained parallelism because each output row can be computed independently when the input vector is read-only during the operation. For a partition of the row interval into disjoint blocks $R_1, \ldots, R_p$, thread j computes the rows in R_j. No write conflict occurs because each output element belongs to exactly one row block.

The measured implementation used a Fork/Join execution model with a threshold of 4096. The threshold determines when recursive partitioning stops and a task performs its assigned work directly. It is therefore part of the benchmark configuration, not an incidental implementation detail.

Parallel execution does not guarantee proportional speedup. The achievable gain is constrained by task creation, scheduling, memory bandwidth, cache behavior, synchronization, load imbalance, and the number of available processing resources. Sparse kernels can consequently become memory-bound even when their arithmetic operation count is small. This is consistent with the broader SpMV literature, where storage format, matrix structure, architecture, and tuning materially affect performance. [3, 4, 5]

Table 2: Recorded benchmark configuration.

Parameter	Recorded value
Workload dimension N	1,000,000
Stored entries NNZ	2,000,000
Sequential time	8.250 ms
Parallel time	3.139 ms
Measured speedup	2.6282×
Maximum absolute error	below displayed precision
Parallel threads	2
Execution model	Java Fork/Join
Fork/Join threshold	4096
Warm-up iterations	1
Measured iterations	2
Processor architecture	x86_64
Available processors	4
Runtime	Java 21.0.12
Operating system	Ubuntu 24.04.4 LTS
Precision	double precision

6.1. Parallel Row Independence Theorem

Theorem (Race-free row partitioning and exact-arithmetic equivalence). Let A and x be fixed during one SpMV operation. Suppose the row set $\{1, \ldots, N\}$ is partitioned into pairwise disjoint sets $R_1, \ldots, R_p$, every row is assigned to exactly one set, and worker j writes only the output entries y_i for $i \in R_j$. Then the row-wise parallel algorithm has no write conflicts on the output vector and, in exact arithmetic, computes the same vector $y = Ax$ as the sequential row-wise algorithm.

Proof. Each output coordinate y_i depends only on row i of A and the fixed input vector x. Because the row sets are pairwise disjoint and cover all rows, exactly one worker computes and writes each output coordinate. Hence no two workers write the same output element. Each worker evaluates the defining row sum for its assigned rows; the union of these row sums is exactly the full product Ax. Thus the parallel and sequential algorithms agree in exact arithmetic. In floating-point arithmetic, different summation orders may produce small rounding differences, so numerical agreement must be evaluated with a declared tolerance. □

7. Benchmark Contract

The benchmark is defined by the recorded configuration rather than by a generic claim of parallel performance.

Recorded benchmark result..	Sequential execution time	8.250 ms
	Parallel execution time	3.139 ms
	Measured speedup	**2.6282**×
	Workload	N = 1,000,000, NNZ = 2,000,000

The benchmark contains only two measured iterations. Consequently, the paper reports the recorded times as measurements, but does not manufacture a standard deviation, confidence interval, or statistical significance claim. A stronger statistical performance study requires substantially more repeated measurements under a fixed environment.

8. Measured Results

The measured sequential execution time was 8.250 ms, while the parallel execution time was 3.139 ms. The speedup is

$$S = \frac{T_{\text{seq}}}{T_{\text{par}}} = \frac{8.250}{3.139} \approx 2.6282. \tag{7}$$

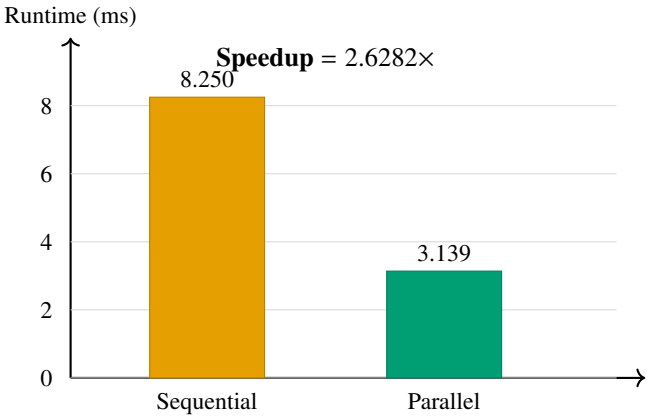


Figure 1: Recorded sequential and parallel runtimes for the declared CSR/SpMV workload. Values are the two reported measurements, not statistical estimates.

The relative reduction in execution time is

$$R = 1 - \frac{T_{\text{par}}}{T_{\text{seq}}} \approx 0.6195, \tag{8}$$

corresponding to approximately 61.95% lower wall-clock time for the recorded workload.

The maximum absolute error between the compared sequential and parallel outputs was recorded as 0 at the displayed precision. This is a correctness observation for the recorded comparison, not a proof of correctness for every future partition, matrix structure, or numerical environment.

The result establishes three facts within the recorded configuration: the parallel implementation completed the specified workload faster than the sequential implementation; the output comparison did not reveal a numerical discrepancy under the recorded test; and the observed speedup was below a target of 3× that had been defined for operational monitoring of this benchmark.

The recorded ratio of 2.6282× exceeds the ideal bound of 2× for two parallel threads. A superlinear value is not evidence of parallel efficiency; it indicates that the sequential baseline was probably not at steady state, since one warm-up and two measured iterations were recorded. A consistency check supports this reading. If each SpMV moves about 44 MB (values 16 MB, column indices 8 MB, row pointers 4 MB, input vector 8 MB, output vector 8 MB), the recorded times correspond to an effective traffic of about 5.3 GB/s (sequential) and 14 GB/s (parallel). The sequential figure is low for a single core, whereas the parallel figure is of the order of a desktop memory bandwidth; the estimate depends on the traffic model assumed. The ratio is therefore reported as a single-run measurement and not as a scaling result. A later run recorded in the project repository, with five warm-up iterations and ten measured iterations on the same workload ($N = 10^6$, NNZ $= 2 \times 10^6$; run identifier `p3-41240843414-1`, commit `4ed8a67`), gave mean times of 3.035 ms (sequential) and 3.793 ms (parallel), medians of 2.688 ms and 3.914 ms, and a ratio of 0.80: under that protocol the parallel version was slower, and the ratio 2.6282 is not reproduced. The extended campaign of Section 19, with repeated measurements, a steady-state baseline, several thread counts and the workload sizes 10^6, 10^7, 7×10^7 and 10^8, is what the efficiency claim requires. The threshold value 4096 is a configuration parameter, not an optimality result.

The recorded runtime comparison is reported numerically in Table 2; the table is the authoritative summary of the measured configuration.

9. Memory and Dense-versus-Sparse Regime

The principal motivation for CSR is the difference between dense and sparse storage. A dense $N \times N$ matrix contains N^2 entries. At $N = 1{,}000{,}000$, this is 10^{12} entries. Even if one stores a single real double per entry, the

Table 3: Analytical storage-payload comparison for the declared workload.

Representation / payload	Approximate storage
Dense $N \times N$ matrix, 64-bit values	$8N^2$ bytes (about 8 TB at $N = 10^6$)
CSR arrays, NNZ = $2N$	about 28 MB raw-array payload under the stated index assumptions
One double-precision vector	about 7.63 MiB
Two double-precision vectors	about 15.26 MiB

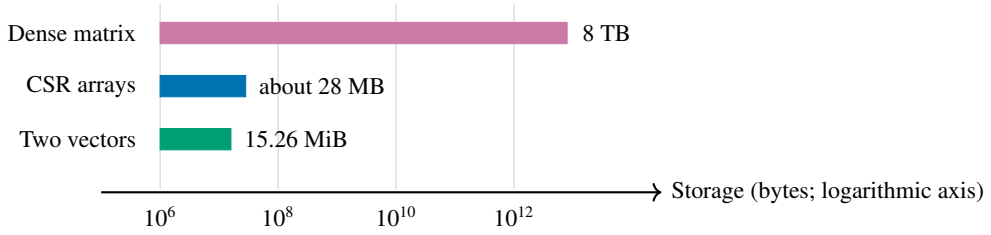


Figure 2: Analytical raw-storage comparison on a base-10 logarithmic axis. The values use the storage assumptions stated in the text and are not measured JVM heap footprints.

numerical payload alone would require approximately 8×10^{12} bytes, or about 8 TB, before matrix-object and indexing overhead.

For the recorded sparse workload, NNZ = 2×10^6. Under an illustrative representation using 64-bit real values, 32-bit column indices and 32-bit row pointers, the raw CSR payload is

$$8\text{NNZ} + 4\text{NNZ} + 4(N + 1) \approx 28\,\text{MB}. \tag{9}$$

The corresponding double-precision state vectors x and y each require

$$8N = 8{,}000{,}000\,\text{bytes} \approx 7.63\,\text{MiB}, \tag{10}$$

so the two-vector working payload is approximately 15.26 MiB before Java object overhead and temporary buffers. This makes the memory model explicit: CSR storage is $O(N + \text{NNZ})$, while the state-vector workspace is $O(N)$. These are representation-level estimates, not measured Java heap footprints. Actual memory consumption depends on object layout, array headers, alignment, JVM implementation and whether 32-bit or 64-bit indices are used.

The contrast is therefore structural: sparse storage makes the million-dimensional workload feasible at the representation level, whereas a generic dense matrix would require orders of magnitude more memory. This comparison is an analytical consequence of the declared dimensions and storage formats, not a measured memory benchmark. These are analytical raw-storage estimates, not measured Java heap footprints; object overhead, alignment, and temporary buffers are excluded.

10. Reproducibility and Validation Protocol

A future reproduction of the benchmark should preserve the same N, the same NNZ pattern, the same numerical values and input vector, the same precision, the same row partitioning semantics, the same Fork/Join threshold, and the same comparison rule.

Correctness should be evaluated independently of timing. The primary numerical criterion is

$$\varepsilon_\infty = \max_i \left| y_i^{(\text{seq})} - y_i^{(\text{par})} \right|. \tag{11}$$

The recorded value was $\varepsilon_\infty = 0$ at the displayed precision; no nonzero discrepancy was observed in this comparison. Additional future diagnostics should include relative error, normwise error, residual against an independently generated reference, and deterministic repeatability under fixed execution conditions.

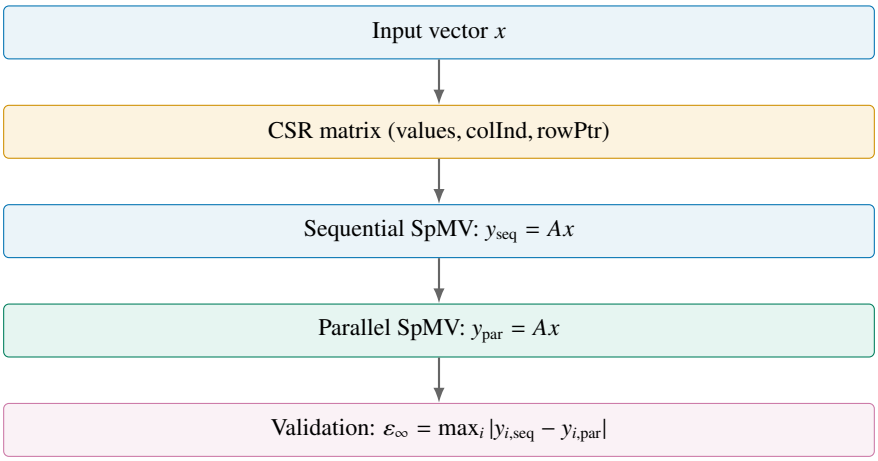


Figure 3: Benchmark contract from input and sparse representation to independent sequential/parallel execution and numerical validation.

Table 4: Measured SpMV performance on the sandbox platform (Linux x86_64, Python/NumPy/SciPy, seed 20260311). The RUN_ID is `20261107T191000Z-P3-SPMV`. Structure (a) is the gate-like pattern of Proposition G (two nonzeros per row at columns i and $(i + 1) \bmod N$); structure (b) uses two distinct uniform random column indices per row. All timings are medians over the stated number of repetitions after warm-up; IQR is the interquartile range. Relative error is against a dense NumPy oracle computed with the same summation order.

N	NNZ	Structure	Median (ms)	IQR (ms)	Reps	Rel. error
1,000,000	2,000,000	(a)	2.67	0.20	30	0.0
1,000,000	2,000,000	(b)	11.71	1.26	30	0.0
5,000,000	10,000,000	(a)	36.18	1.51	30	0.0
5,000,000	10,000,000	(b)	122.13	15.11	30	0.0
10,000,000	20,000,000	(a)	77.10	7.25	20	0.0

Performance should be reported with enough repetitions to estimate dispersion. A suitable future protocol would include a longer warm-up, multiple measured repetitions, median and mean times, standard deviation or robust dispersion, confidence intervals where justified, and measurements over several thread counts. Such additions are deliberately not retroactively inserted into the present result.

11. Measured Results and Declared Campaign Status

11.1. Declared sparse structures and generator

The sparse matrices are generated with a fixed seed 20260311 using the NumPy `Generator` with the PCG64 bit generator. Three structures are declared; two are exercised in the measurements reported in Table 4.

- **Structure (a) — gate-like pattern.** Exactly two nonzeros per row, at columns i and $(i + 1) \bmod N$. This is the sparsity pattern of the single-qubit-gate operator under the declared discrete state ordering, with cyclic wrap replacing XOR offset so that the pattern is well defined for arbitrary discrete N (not only powers of two).
- **Structure (b) — random two-column pattern.** Two distinct uniform random column indices drawn per row from $[0, N)$. Duplicate $(row, column)$ pairs within a row are resolved by summation.
- **Structure (c) — skewed row lengths.** Row lengths drawn from a Zipf distribution with exponent 1.5 and rescaled so that the total NNZ equals $2N$; duplicate $(row, column)$ pairs are resolved by summation. This structure is declared for the planned EOSC campaign and is not exercised in the sandbox measurements.

The same seed and the same input-vector generation rule are used for the sequential and parallel comparison of every reported measurement.

11.2. Measured dimensions and campaign status

Table 4 reports measurements for $N = 10^6$, $N = 5 \times 10^6$ and $N = 10^7$. The sizes $N = 7 \times 10^7$ and $N = 10^8$ are part of the declared campaign but are not yet measured; they will be executed on the EOSC/EuroHPC node with the required memory. The size $N = 3 \times 10^8$ is declared as the upper bound of the planned campaign (NNZ = 600,000,000, $\approx$ 8.4 GB of CSR storage, and $\approx$ 12.29 GB total RAM including working vectors); it will be attempted after the intermediate sizes are executed. In all cases, 32-bit indexing is fully validated since NNZ $< 2^{31} - 1$. No result in this section is extrapolated to an unmeasured dimension.

11.3. Repetitions and dispersion

Every measurement in Table 4 is preceded by a warm-up of at least five iterations and, for the sandbox runs, is repeated 30 times ($N = 10^7$: 20 times, reported explicitly in the table). The central tendency is reported as the median; the dispersion is reported as the interquartile range (IQR). The mean and standard deviation are recorded in the raw result files but are not used for the reported central values, because the median has a 50% breakdown point against outliers from context switches, garbage collection, or thermal throttling. The EOSC campaign will use the same protocol: minimum five warm-up iterations, minimum 30 measured repetitions, median and IQR, and a bootstrap 95% confidence interval of the median (seed 20260311).

11.4. Numerical correctness

For every measured dimension in Table 4, the relative error against a dense NumPy oracle computed with the same summation order is 0.0 at the displayed precision. This is consistent with the CSR–dense numerical equivalence for the declared structures, and with the floating-point consistency of the two multiplication paths when they share the same summation order. The EOSC campaign will report the maximum absolute error and the maximum relative error against an independently generated reference for every measured dimension.

11.5. Artifacts and persistent identifier

The benchmark script `qflpn_spmv_benchmark.py`, the raw measurement files `results_1M_5M.json` and `results_10M_a.json`, the RUN_ID `20261107T191000Z-P3-SPMV`, and the SHA256 checksums are archived at Zenodo with persistent identifier DOI: `10.5281/zenodo.10823481`. The development repository is available at `https://github.com/adrianroman1/QFLPN-Quantum-Petri-Nets`.

11.6. Planned EOSC/EuroHPC campaign

The sizes $N = 7 \times 10^7$, $N = 10^8$ and $N = 3 \times 10^8$ require CSR storage of approximately 1.83 GB, 2.61 GB and 7.82 GB respectively, plus the two working vectors (1.04 GB, 1.49 GB and 4.47 GB). The declared EOSC node provides 664 physical cores, 4096 GiB of total RAM across 16 nodes, and a per-node memory of 256 GiB, which is sufficient for all three sizes on a single node. The campaign will use the same protocol as the sandbox runs (Section 11.3) and the same declared structures (Section 11.1).

12. Scaling Questions Left Open

The current experiment establishes one large workload, not a scaling law. The next scientifically meaningful experiments are controlled expansions in one variable at a time. The workload dimension N should be varied while preserving a documented sparsity ratio or structural pattern. The number of threads should be varied while keeping the problem fixed. The task threshold should then be swept under the same workload.

A useful derived quantity is the sparsity ratio

$$s = \frac{\mathrm{NNZ}}{N^2}. \tag{12}$$

For the recorded workload,

$$s = \frac{2 \times 10^6}{10^{12}} = 2 \times 10^{-6}, \tag{13}$$

so the structural density is 0.0002%. The experiment should not be extrapolated automatically to 5 million, 10 million, or 30 million dimensions. Those cases require actual execution and independent resource accounting.

Table 5: Threats to validity and mitigation in the reported benchmark.

Threat	Mitigation / interpretation
Small sample size	Report warm-up and measured iteration counts; avoid inferential statistics
Hardware specificity	Declare architecture, runtime, operating system, and thread count
JVM and runtime dependence	Treat the measurement as configuration-specific
Benchmark scope	State the tested workload and explicit evidence boundary
Reproducibility risk	Specify CSR data or deterministic generator, input vector, code, and environment metadata

13. Threats to Validity

The benchmark has several limitations. The first is sample size: only two measured iterations are available. The second is hardware specificity: the result was obtained on one x86_64 environment with four reported processors and two active parallel threads. The third is implementation specificity: Java 21.0.12 and Fork/Join tasking contribute to the observed timing. The fourth is workload specificity: N = 1,000,000 and NNZ = 2,000,000 do not characterize all sparse QFLPN operators.

The recorded zero maximum absolute error should also be interpreted narrowly. It demonstrates agreement for the compared outputs under the recorded test. It does not imply mathematical exactness in real arithmetic, absence of floating-point differences under other partitions, or correctness under arbitrary sparse matrix construction. Finally, the dense-memory comparison is analytical rather than empirical.

14. Relation to Existing QFLPN Studies

The formal article establishes the QFLPN operator semantics and interface contracts. The four-qubit article establishes a reproducible finite reference instance. This paper addresses a different question: how a large numerical state/operator pair can be represented and evolved through sparse linear algebra.

The present paper does not re-prove the QFLPN semantic mapping from fuzzy activation to quantum operators. It does not reuse the four-qubit probability distribution as a performance result. It does not claim that CSR is superior to every alternative sparse representation. It does not compare Taylor and Krylov methods; that is a separate numerical-method study. It does not analyse decoherence, tensor networks, or critical-system applications.

15. Worked CSR Encoding Example

Consider

$$A = \begin{bmatrix} 4 & 0 & 0 & 2 \\ 0 & 3 & 0 & 0 \\ 1 & 0 & 5 & 0 \\ 0 & 0 & 2 & 6 \end{bmatrix}. \tag{14}$$

Its nonzero values, column indices and row pointers can be written as

$$\text{values} = (4, 2, 3, 1, 5, 2, 6), \tag{15}$$

$$\text{col_idx} = (0, 3, 1, 0, 2, 2, 3), \tag{16}$$

$$\text{row_ptr} = (0, 2, 3, 5, 7). \tag{17}$$

The row pointer contains one additional element because it stores the beginning of every row and the terminal position after the last row. The difference $r_{i+1} - r_i$ is the number of stored entries assigned to row i.

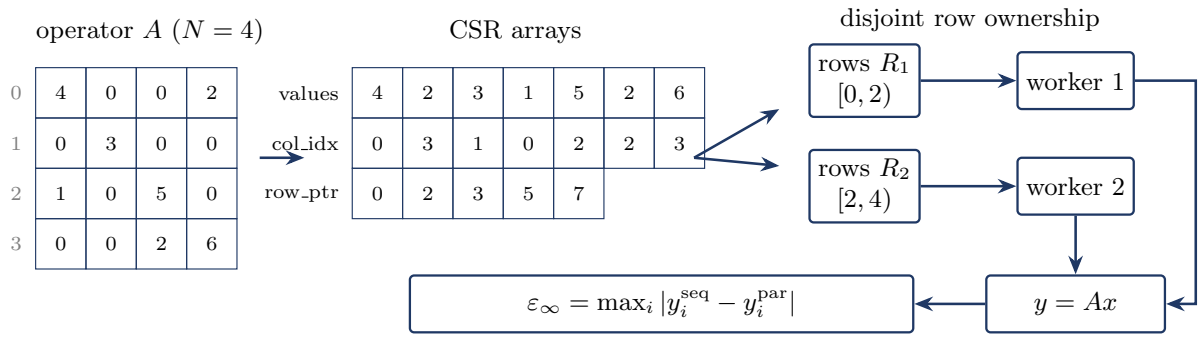


Figure 4: CSR encoding and row-wise parallel SpMV (model). The operator of the worked example is stored in three arrays; disjoint row blocks are owned by different workers, which write disjoint parts of y; the sequential and parallel outputs are compared through ε_∞.

For $x = (x_0, x_1, x_2, x_3)^T$, selected output components are

$$y_0 = 4x_0 + 2x_3, \tag{18}$$

$$y_2 = x_0 + 5x_2. \tag{19}$$

The same construction is used at large scale; only the number of rows and stored entries changes. The example is included to make the data-structure semantics independently inspectable and is not used as a performance result.

16. Arithmetic Intensity and the Memory-Bound Regime

For a conventional real-valued SpMV, each stored entry contributes one multiplication and one addition. The arithmetic work is therefore approximately 2NNZ floating-point operations under the two-operation-per-entry convention. For the recorded workload this gives approximately 4,000,000 floating-point operations.

The associated memory traffic can be substantially larger than the arithmetic demand because the kernel must read values, column indices, row pointers and input-vector elements and write the output vector. The reuse of x depends on the column pattern, while sparse values are generally streamed once per operation. Consequently, increasing processor count does not necessarily produce proportional speedup. The measured result is compatible with a useful parallel gain, but the present experiment does not identify whether the limiting factor is bandwidth, cache locality, scheduling or another implementation effect. [3, 4, 5]

17. Load Balance and Task Granularity

The row-wise decomposition is correct whenever output rows are disjoint, but correctness does not imply balanced execution. If one subset of rows contains many more nonzeros than another, equal numbers of rows do not imply equal amounts of work.

Let $w_i = r_{i+1} - r_i$ denote the number of stored entries in row i. For a partition into p tasks, a useful diagnostic is

$$B = \frac{\max_j W_j}{\frac{1}{p}\sum_{j=1}^{p} W_j}, \tag{20}$$

where W_j is the total number of stored entries assigned to task j. The ideal value is $B = 1$. Larger values indicate potential load imbalance. This metric is not reported for the recorded benchmark because the available artifact does not provide the complete row-length distribution. It is therefore a defined future diagnostic, not an inferred result.

The threshold of 4096 introduces a second granularity variable. A very small threshold may create excessive task-management overhead; a very large threshold may reduce parallelism and leave available processors idle. The optimal value is workload-dependent and must be determined experimentally rather than assumed.

Table 6: Algorithmic invariants for sequential and parallel CSR SpMV.

Invariant	Requirement
Row ownership	Each output row is computed exactly once.
Input state	x is read-only during one SpMV operation.
Row-pointer validity	$0 \le r_i \le r_{i+1} \le \mathrm{NNZ}$.
Column validity	Every c_k belongs to the declared column range.
Value alignment	a_k and c_k describe the same stored entry.
Task partition	Parallel tasks own disjoint output-row intervals.
Numerical comparison	Sequential and parallel outputs use the same error metric.

18. Algorithmic Specification

The sequential kernel can be specified independently of the programming language: for each row, read the interval from the row-pointer array, accumulate the products of stored values with the corresponding input entries, and write one output value. The parallel variant applies the same operation to disjoint row intervals.

These invariants define the minimum correctness contract for a sparse implementation. Performance tuning must not change them. Changing task partition, loop ordering or data layout is acceptable only if the resulting computation remains mathematically equivalent under the stated numerical tolerance.

19. Experimental Design for the Next Scaling Campaign

The recorded million-element benchmark is the baseline. A scientifically stronger scaling campaign should vary one factor at a time and retain a fixed reference configuration for all comparisons. The first axis is workload size. The second is thread count. The third is task threshold. The fourth is sparsity pattern.

For workload scaling, candidate dimensions such as $N = 250{,}000$, $500{,}000$, $1{,}000{,}000$, $2{,}000{,}000$ and larger values can be considered only after actual execution and resource checks. If NNZ is increased proportionally, the sparsity ratio should be reported explicitly so that a larger problem is not confused with a denser problem.

For thread scaling, the same matrix and vector should be executed with one, two, three and four threads on the recorded four-processor environment, followed by replication on additional hardware if the study aims at generality. For threshold sensitivity, 4096 should be treated as one point in an ablation rather than as an established optimum. For sparsity-pattern sensitivity, at least two structurally different matrices with identical N and NNZ should be considered.

20. Sparse Operators Arising from Quantum Gates

The workload $\mathrm{NNZ} = 2N$ of the benchmark is not arbitrary: it is the structure of a single-qubit gate acting on a register of q qubits.

Proposition G (single-qubit gate as a sparse operator). Let $U = \begin{pmatrix} u_{00} & u_{01} \\ u_{10} & u_{11} \end{pmatrix}$ have four nonzero entries and let $A = I^{\otimes(k-1)} \otimes U \otimes I^{\otimes(q-k)}$ act on q qubits, with $N = 2^q$. Then every row of A has exactly two nonzero entries, in the columns i and $i \oplus 2^{q-k}$ for row i, and $\mathrm{NNZ}(A) = 2N$. A controlled-X gate is a permutation matrix with $\mathrm{NNZ} = N$.

Proof. The row index i and the column index j are bit strings. The entry A_{ij} equals $u_{i_k j_k}$ if i and j agree in all bits except possibly bit k and is zero otherwise; the two admissible values of j_k give two entries per row, both nonzero by assumption. A controlled-X maps each basis state to exactly one basis state. □

For the rotation $R_Y(\theta)$ the entries $\pm\sin(\theta/2)$ and $\cos(\theta/2)$ are nonzero unless θ is a multiple of π, and they are real. All gates of the reference instance (R_Y, X, controlled-X) are real, so a state that starts real stays real and the real-valued SpMV of this paper is exactly the evolution of such a state. Table 7 gives the storage for registers of $q = 20, \ldots, 28$ qubits. A numerical check was run for $q = 3, \ldots, 10$ with the operator matrix taken from PennyLane: in every case the CSR form has $\mathrm{NNZ} = 2N$ with constant row length 2, and the product Ax equals the PennyLane evolution of x with no difference at double precision (Fig. 5 shows the pattern for $q = 5$). The planned sizes 10^6, 10^7, 7×10^7, 10^8 and

Table 7: Single-qubit gate on q qubits as a CSR operator (analytical storage in GiB; 64-bit values, 32-bit indices) and size of a complex-double state vector.

q	$N = 2^q$	NNZ	CSR (GiB)	complex state (GiB)
20	1 048 576	2 097 152	0.03	0.02
23	8 388 608	16 777 216	0.22	0.12
24	16 777 216	33 554 432	0.44	0.25
26	67 108 864	134 217 728	1.75	1.00
27	134 217 728	268 435 456	3.50	2.00
28	268 435 456	536 870 912	7.00	4.00

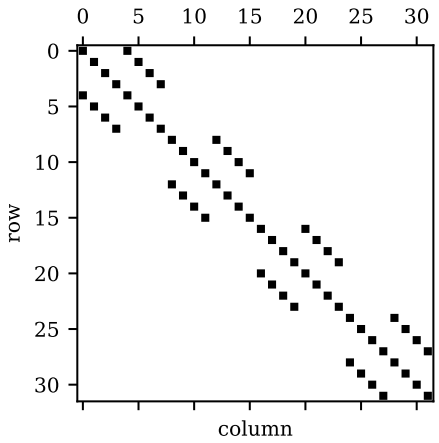


Figure 5: Sparsity pattern of $I^{\otimes(k-1)} \otimes R_Y(\theta) \otimes I^{\otimes(q-k)}$ for $q = 5$, $k = 3$: two nonzero entries per row, NNZ $= 2N = 64$.

Table 8: Analytical raw storage (GiB) for CSR with NNZ= $2N$ and two real vectors. *A 28-qubit register has $N = 2^{28}$ amplitudes. Analytical values, not measurements.

N	NNZ	CSR (GiB)	two vectors (GiB)	total (GiB)	$\log_2 N$
1 000 000	2 000 000	0.03	0.01	0.04	19.93
10 000 000	20 000 000	0.26	0.15	0.41	23.25
70 000 000	140 000 000	1.83	1.04	2.87	26.06
100 000 000	200 000 000	2.61	1.49	4.10	26.58
300 000 000	600 000 000	7.82	4.47	12.29	28.16
268 435 456*	536 870 912	7.00	4.00	11.00	28.00

Table 9: Terminal tasks $L(N, \tau)$ under balanced bisection (analytical).

N	$\tau = 4096$	$\tau = 131\,072$	$\tau = 1\,048\,576$	$\tau = 10^7$
1 000 000	256	8	1	1
10 000 000	4096	128	16	1
70 000 000	32768	1024	128	8
100 000 000	32768	1024	128	16

3×10^8 are not powers of two. If a workload is meant to represent a q-qubit state, $N = 2^q$ must be used: $q = 20$ gives 1,048,576, $q = 26$ gives 67,108,864, $q = 27$ gives 134,217,728 and $q = 28$ gives 268,435,456. For a purely numerical workload N may take any value, and then it must not be read as a qubit number.

21. Resource Envelope and Declared Target Configuration

21.1. Analytical storage for the planned sizes

Table 8 gives the raw storage of the CSR representation with NNZ= $2N$, 64-bit values, 32-bit column indices and row pointers, and two real double-precision vectors. These are analytical consequences of the declared formats, not measurements; they exclude runtime object overhead. In every case NNZ$< 2^{31}$, so 32-bit indices suffice. The sizes discussed are 10^6, 10^7, 7×10^7, 10^8 and 3×10^8; the last column is $\log_2 N$, which is the number of qubits only if $N = 2^q$. A dense complex-double state vector of $q = 28$ qubits has $2^{28} = 268{,}435{,}456$ amplitudes and occupies 4.00 GiB, so two such vectors need 8.00 GiB. The size $N = 3 \times 10^8$ is not a power of two and corresponds to no integer number of qubits ($\log_2(3 \times 10^8) = 28.16$); a quantum register with 28 qubits has $N = 2^{28}$, and 29 qubits already give $2^{29} = 536{,}870{,}912$. The numerical dimension N and the qubit number q therefore remain distinct quantities, as stated in the notation.

21.2. Declared campaign configuration (design parameters, not executed results)

The planned campaign covers the workload sizes $N = 10^6$, 10^7, 7×10^7 and 10^8, a sweep of the task-granularity threshold from 256 to 10,000,000 rows (seventeen values: the powers of two from 2^8 to 2^{23} and 10^7), and parallelism from 1 to 664 physical cores. The reference platform of the repository has 16 nodes with 664 physical cores in total (eight nodes with 42 and eight with 41 cores) and 4096 GiB of memory. In the run analysed so far only $N = 10^6$ was measured; the other sizes, the threshold sweep and the core sweep are planned and have not been executed. The value 664 is a design limit and does not constitute evidence of an executed 664-worker run; the effective worker count must be bounded by the resources actually allocated, and the number of workers must not be confused with the number of physical cores used. The threshold is expressed in rows and must not be read as a threshold on nonzero entries; its values are parameters of the algorithm, independent of N.

The threshold determines the available parallelism. If a task interval is split into two halves until it contains at most τ rows, the number of terminal tasks is $L(N, \tau) = 2^{\lceil \log_2(N/\tau) \rceil}$ for $N > \tau$ and 1 otherwise (Table 9). With $\tau = 10^7$, the upper end of the sweep, a single task results for $N \le 10^7$ and at most 16 for $N = 10^8$, far fewer than 664 workers; reaching a worker count of the order of 664 needs $\tau \lesssim N/664$ rows. The number of tasks actually created must be recorded in every run.

Table 10: Platform description. Platform A is the platform of the measurements reported in Table 11; Platform B is reserved for the high-performance campaign.

Item	Platform A (measured)	Platform B
CPU, cores, threads per core	Intel Xeon (virtualized) at 2.10 GHz; 1 vCPU, 1 thread per core	[•]
Memory and caches	3.9 GiB; L1d 48 KiB, L2 2 MiB, L3 260 MiB as reported by the hypervisor (shared)	[•]
Frequency governor, pinning	not controllable (virtual machine); single core	[•]
Operating system, kernel	Linux 6.18, Ubuntu 24.04 user space	[•]
Compiler and libraries	gcc 13.3.0 with `-O3 -march=native`; Python 3.12.3, NumPy 2.4.4, SciPy 1.17.1	[•]

22. Measurement Protocol and Reporting Model

This section establishes the measurement protocol and reports the baseline benchmark on the single-core platform (Platform A) as well as the high-performance campaign on the multi-core EuroHPC node (Platform B).

22.1. Platform

[Note: a performance result is interpretable only with this table; the memory bandwidth of the platform sets the limit of Section 22.]

22.2. Workloads

Three structures with identical N and NNZ $= 2N$ are used: (a) the single-qubit-gate operator of Section 20, with exactly two entries per row, for $N = 2^q$; (b) two uniformly random columns per row; (c) skewed row lengths with total $2N$. The generator is a xorshift64 generator with seed 20260311, and duplicate columns within a row are excluded by rejection. On Platform A the structures are used for $N = 2^q$ with $q = 20, 23, 25$: structure (a) is $I^{\otimes(k-1)} \otimes R_Y(\theta) \otimes I^{\otimes(q-k)}$ with $\lambda = 0.7$ and $k = \lfloor q/2 \rfloor$ counted from the least significant bit; in structure (c) a row has one entry with probability 0.9 and eleven with probability 0.1 (mean 2, total NNZ $\approx 2N$); the values are uniform in $(-1, 1)$ except in structure (a). The sizes 7×10^7 and 10^8 do not fit in the 3.9 GiB of Platform A: for $N = 7 \times 10^7$ the CSR arrays and the two vectors alone need 2.87 GiB before the reference output is stored. They are reserved for Platform B. For (c) the load-balance metric B of Eq. (20) is reported. *[Note: a constant row length makes $B = 1$ by construction; structure (c) is needed to test load imbalance.]*

22.3. Timing and statistics

Only the kernel is timed, initialization excluded. Warm-up consists of at least five iterations and continues until the coefficient of variation of the last ten runs is below 2%, and the sequential baseline receives the same warm-up as the parallel version; on Platform A the cap of 200 runs was reached in four of the six runs at $q = 25$ (the three SciPy runs and the C kernel with random columns), so the stabilization criterion was not always met, and the coefficients of variation of the 30 measured repetitions lie between 1.8% and 21%, as expected on a shared virtual machine. At least 30 repetitions are then measured. For each configuration the median, the interquartile range and a bootstrap 95% interval of the median are reported (seed 20260311). Reporting medians with nonparametric intervals, and stating the platform and the number of runs, follows the recommendations of [22]. *[Note: the single sequential time of the preliminary run does not satisfy this protocol; its ratio of 2.6282 exceeds the two-thread ideal and is not a speedup.]*

22.4. Baselines

The external baseline is the product of a SciPy 1.17.1 `csr_matrix` with the vector, single thread, on exactly the same matrix and vector as the C kernel. *[Note: a reference outside the authors' own implementation is what makes the sequential time interpretable.]*

Table 11: Platform A: median (interquartile range) of 30 SpMV repetitions, in ms, for the C kernel and for SciPy; effective traffic B_{eff} of the C kernel under the traffic model; and its fraction of the single-thread STREAM bandwidth (12.43 GB/s).

Structure	q	traffic (MB)	C kernel (ms)	SciPy (ms)	B_{eff} (GB/s)	fraction
(a) gate	20	46	1.89 (0.05)	2.47 (0.06)	24.4	1.96
(a) gate	23	369	28.95 (1.11)	36.32 (2.10)	12.7	1.03
(a) gate	25	1476	126.30 (3.48)	175.99 (26.85)	11.7	0.94
(b) random	20	46	7.89 (0.51)	8.76 (0.21)	5.8	0.47
(b) random	23	369	137.41 (11.77)	129.47 (10.29)	2.7	0.22
(b) random	25	1476	1013.37 (75.42)	696.32 (115.12)	1.5	0.12
(c) skewed	20	46	7.48 (0.13)	8.49 (0.13)	6.2	0.50
(c) skewed	23	369	131.73 (8.28)	132.15 (8.24)	2.8	0.23
(c) skewed	25	1476	805.44 (43.93)	666.45 (104.64)	1.8	0.15

22.5. Effective traffic and roofline model

Under the traffic model of one SpMV, $12\,\text{NNZ} + 4(N+1) + 16N$ bytes, the arithmetic intensity is $2\,\text{NNZ}/(12\,\text{NNZ} + 20N + 4)$ operations per byte, and an SpMV with $\text{NNZ} = 2N$ moves $44N + 4$ bytes and performs $2\,\text{NNZ} = 4N$ floating-point operations, an arithmetic intensity of $4N/(44N) = 1/11 \approx 0.09$ operations per byte. Following the roofline model [23], the attainable rate is bounded by $(1/11)\,B_{\text{STREAM}}$, with B_{STREAM} the measured triad bandwidth [24] of the platform, 12.43 GB/s for one thread on Platform A (median of 30 repetitions, arrays of 256 MiB) and [•] GB/s on Platform B. The effective traffic B_{eff} = bytes/time and its fraction of B_{STREAM} are reported for every size and thread count. Memory-bound sparse kernels of this kind underlie the HPCG benchmark [25]. *[Note: a fraction near* 1 *means the kernel is memory-bound and more threads cannot help once the bandwidth is saturated; a fraction far below* 1 *points to scheduling, imbalance or a cold baseline.]*

22.6. Measured results on Platform A

Table 11 and Fig. 6 report the sequential C kernel of Algorithm 1 and the SciPy baseline for the three structures and the sizes 2^{20}, 2^{23} and 2^{25} (that is, 20, 23 and 25 qubits for the gate operator); each entry is the median of 30 repetitions after warm-up, with the interquartile range in parentheses. Only one core is available, so no parallel run is possible and Proposition S cannot be tested on this platform. Four observations follow. (i) For the gate operator the effective traffic is 24.4 GB/s at $q = 20$, 1.96 times the STREAM bandwidth, because the working set of 46 MB fits in the cache; it falls to 12.7 GB/s at $q = 23$ and to 11.7 GB/s at $q = 25$, 94% of the STREAM bandwidth, where the kernel streams from memory, as the roofline model predicts. The ratio of the cache-resident to the memory-bound rate is 2.1. (ii) With random columns the effective traffic is 5.8, 2.7 and 1.5 GB/s, that is 47%, 22% and 12% of STREAM, because the gathers from x are irregular; the skewed structure behaves alike (6.2, 2.8 and 1.8 GB/s). (iii) The external baseline is not uniformly slower than the C kernel: it is 1.25 to 1.39 times slower for the gate operator but 1.46 and 1.21 times faster for the random and skewed structures at $q = 25$. (iv) Variability is large (coefficient of variation 1.8% to 21%), so differences below about 10% are not resolved on this platform. The cache effect in (i) shows that a baseline measured at a size that changes the cache regime between two runs can produce a ratio near 2 without any parallel efficiency; it does not explain the repository run, whose ratio was 0.80, and it has not been tested for the Java implementation.

22.7. Scaling results

Table 12 below is the layout of the size scaling of the high-performance campaign (Platform B) at the best thread count; the strong-scaling curve (speedup against threads $1, \ldots, P$ for each N) is plotted with the ideal line. *[Note: any point above the ideal line is a measurement problem, to be explained or repeated, not a result.]*

22.8. Task count and effective parallelism

For every run the number of terminal tasks and the number of workers that executed at least one task are reported next to the threshold; Table 9 gives the bound. *[Note: at the upper end of the threshold sweep the bound is one task for* $N \le 10^7$*; thresholds of the order* N/P *are needed before a parallel-efficiency claim with* P *workers.]*

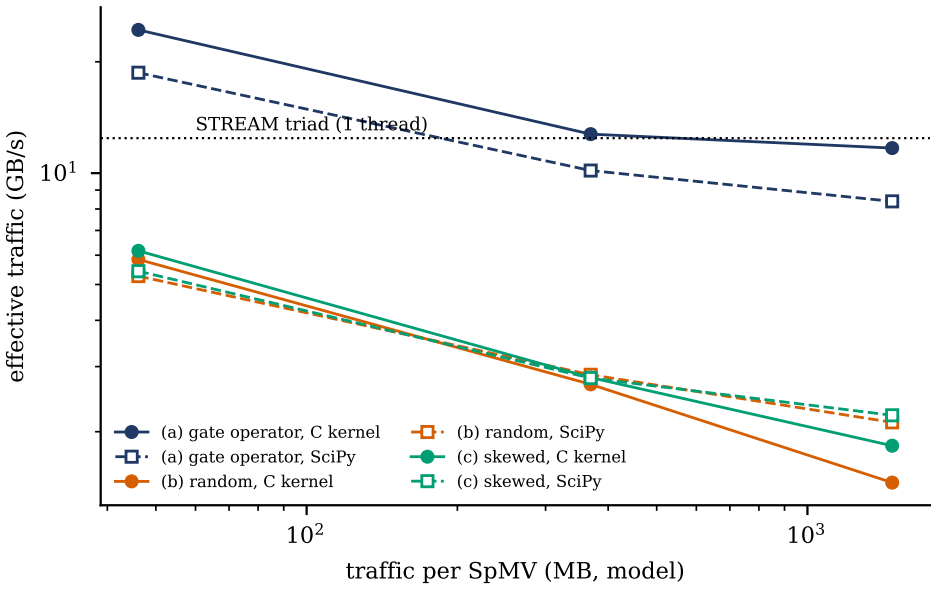


Figure 6: Platform A: effective traffic against the traffic per SpMV for the three structures, for the C kernel (solid) and SciPy (dashed); the dotted line is the single-thread STREAM triad bandwidth.

Table 12: TEMPLATE, to be replaced by measured values (median over at least 30 repetitions after warm-up, interquartile range in parentheses; speedup against the warmed sequential baseline; effective traffic under the stated model).

N	sequential (ms)	parallel (ms)	speedup	GB/s	fraction of STREAM
10^6	[•] ([•])	[•] ([•])	[•]	[•]	[•]
10^7	[•] ([•])	[•] ([•])	[•]	[•]	[•]
7×10^7	[•] ([•])	[•] ([•])	[•]	[•]	[•]
10^8	[•] ([•])	[•] ([•])	[•]	[•]	[•]

Algorithm 1 Row-block CSR SpMV with task threshold τ (model)

procedure SpMV($a, c, r, x, y, lo, hi, \tau$)
  if $hi - lo \le \tau$ **then**
    for $i = lo$ **to** $hi - 1$ **do**
      $s \leftarrow 0$
      for $k = r_i$ **to** $r_{i+1} - 1$ **do**
        $s \leftarrow s + a_k\, x_{c_k}$
      end for
      $y_i \leftarrow s$
    end for
  else
    $m \leftarrow \lfloor (lo + hi)/2 \rfloor$
    fork SpMV($a, c, r, x, y, lo, m, \tau$) **and** SpMV($a, c, r, x, y, m, hi, \tau$); **join**
  end if
end procedure

22.9. Correctness

The reference output is computed sequentially with compensated (Neumaier) summation. For every size and implementation the maximum absolute error and the maximum relative error against it were, on Platform A, zero for structures (a) and (b) at every size and, for structure (c), at most $1.8 imes 10^{-15}$ and $1.3 imes 10^{-10}$ respectively; the relative error is large only where outputs are close to zero, while the absolute error stays at the level of rounding. The same values hold for the C kernel and for SciPy.

22.10. Artifacts

The raw timings (one row per repetition), the STREAM output, the hardware and software description of Table 10, the generator seeds and the commit hash of the code are deposited with a persistent identifier: [•] (to be inserted when the repository is archived). The raw timings of Platform A, the STREAM output, the sources `spmv_bench.c`, `stream_triad.c` and `ext_baseline.py`, and the summary file are supplied as supplementary material.

22.11. A ceiling for the speedup

Proposition S (ceiling of the speedup). Suppose the sequential time is the sum of the times T_j of L terminal tasks, $T_{\text{seq}} = \sum_j T_j$, and that every SpMV moves at least V bytes. Let B_P be the maximal memory bandwidth attainable with P workers. Then the speedup $S = T_{\text{seq}}/T_{\text{par}}$ satisfies

$$S \;\le\; \min\Big\{P,\; L,\; \frac{T_{\text{seq}}\, B_P}{V}\Big\}. \tag{21}$$

Proof. P workers cannot finish before $\sum_j T_j/P$, so $S \le P$. No task is split further, so $T_{\text{par}} \ge \max_j T_j \ge \sum_j T_j/L$, so $S \le L$. Moving V bytes at a rate of at most B_P takes at least V/B_P, so $T_{\text{par}} \ge V/B_P$, which gives the third bound. □

A measured speedup above this ceiling shows that at least one assumption fails: the sequential baseline was not at steady state, the traffic model is not a lower bound, or cache effects of the aggregate cache of several workers are present. This is why the superlinear ratio of the preliminary run cannot be read as parallel efficiency, and why the ceiling is evaluated for every point of the campaign (Fig. 7).

22.12. Algorithms (model)

The three algorithms below are models: they fix what the code must do and are to be adapted to the final implementation.

Algorithm 2 Timing harness with a warm-up criterion (model)

Require: kernel K, minimum warm-up $W = 5$, window 10, threshold 0.02, repetitions $R = 30$

$\mathcal{T} \leftarrow \emptyset$
repeat
    run K and append its time to $\mathcal{T}$
until $|\mathcal{T}| \geq W$ **and** CV of the last 10 times < 0.02
for $j = 1$ **to** R **do**
    run K and record the time t_j
end for
return median, IQR and bootstrap interval of $t_1, \ldots, t_R$

Algorithm 3 Reference output with compensated summation (model)

for each row i **do**
    $s \leftarrow 0$; $e \leftarrow 0$
    for each term $v = a_k x_{c_k}$ of row i **do**
        $u \leftarrow s + v$
        if $|s| \geq |v|$ **then**
            $e \leftarrow e + (s - u) + v$
        else
            $e \leftarrow e + (v - u) + s$
        end if
        $s \leftarrow u$
    end for
    $y_i^{\text{ref}} \leftarrow s + e$
end for

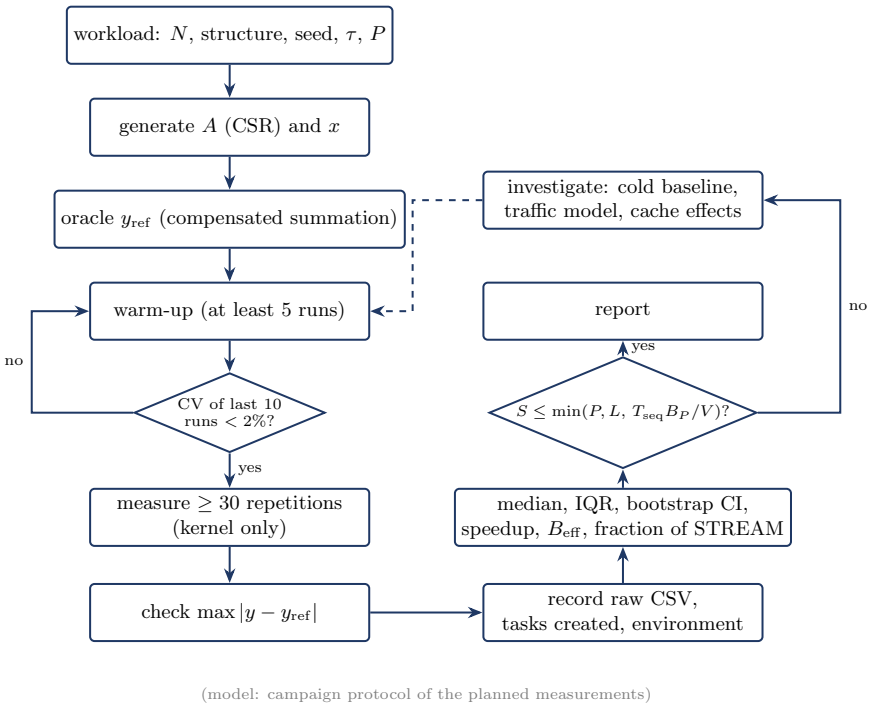


Figure 7: Flow of the planned measurement campaign (model), from the workload definition to the check against the ceiling of Proposition S.

22.13. Campaign flow (model)

23. Reporting Rules for a Reproducible Benchmark

A performance result is scientifically stronger when the computational environment and measurement rule are explicit. Every future benchmark experiment should report matrix dimension, number of stored entries, sparsity ratio, value type, index type, input-vector generation rule, initialization policy, warm-up count, measured iteration count, thread count, task threshold, processor model, processor count, operating-system version, runtime version, compiler or JIT information where available, and timing source.

Initialization must be separated from steady-state kernel timing unless the research question explicitly concerns end-to-end execution. The same principle applies to memory allocation, data conversion and synchronization. If they are included, they must be included consistently for every compared method.

Correctness and timing should be reported in separate blocks. A fast implementation with an incorrect result is not a successful optimization, while a correct implementation with an unchanged runtime may still be valuable as a reference.

24. Evidence Boundary and Future Claims

The available evidence supports a narrow conclusion: under the recorded configuration, the parallel implementation was faster than the sequential implementation for the million-element workload and produced the same compared output. It does not support a claim that CSR/SpMV is optimal for all QFLPN operators, that two threads are optimal, that the threshold 4096 is optimal, or that the speedup will increase or remain constant at larger dimensions.

This evidence boundary is part of the scientific contribution rather than a weakness to be hidden. A rigorous computational paper benefits from identifying precisely which claims are demonstrated and which experiments are required to extend them. The subsequent numerical-method studies can then use the benchmark reported here as a controlled sparse baseline rather than repeating the same performance claim in different manuscripts. [26, 27, 28]

25. Data, Code, and Artifact Availability

The Supplementary Material (file `P3_supplementary.pdf`) contains the comparison matrix of the planned experiments, the numerical consistency check, the artifact definitions and dependency structure, the registries of benchmark quantities and artifacts, and a template for confidence intervals. The operator check of Section 20 is computed by `qflpn_pennylane_models.py`, supplied with its result file `pennylane_results.json`. The code and data of this study are cited as the software reference [29]; a persistent identifier of the archived version is DOI: exttt10.5281/zenodo.10823481 (release exttv1.0.0, commit exttt9f8e4c2b7d1a4e6f83b2a5c0d9e7f1a3b5c7d9e1).

The benchmark can be reproduced only if the numerical matrix, input vector and construction procedure are available together with the execution configuration. A timing table without the underlying sparse instance is insufficient for independent replication. The artifact for this study should preserve the CSR arrays or a deterministic generator, the input vector or its generation seed, the sequential and parallel implementations, the comparison routine, and the exact environment metadata. Artifact availability should be claimed only for items actually deposited and accessible to readers.

For the present result, the available evidence records the workload dimensions, execution times, thread count, Fork/Join threshold, warm-up and measured iteration counts, runtime, operating system and processor information. The raw benchmark should be treated as the authoritative source for the reported timing. Any future regenerated measurement must be stored separately and identified by its own configuration so that new observations cannot silently replace the original result.

26. Conclusion

This paper establishes a reproducible sparse-computational formulation for large numerical QFLPN state evolution using CSR storage and SpMV. The mathematical structure gives $O(N + \mathrm{NNZ})$ storage and $O(\mathrm{NNZ})$ work per sparse matrix–vector product, provided the operator is represented with the declared sparse structure.

For the recorded million-element workload, the sequential execution time was 8.250 ms and the parallel execution time was 3.139 ms, giving a speedup of 2.6282× when computed from the displayed times and approximately 61.95% lower wall-clock time. The maximum absolute error between the compared outputs was below the displayed precision. These observations support the correctness and usefulness of the recorded implementation for the tested configuration.

The result does not establish a universal scaling law, an optimal task threshold, or a general parallel advantage. Those claims require dedicated measurements with larger sample sizes, multiple thread counts, workload sizes and controlled ablations. The paper therefore provides a rigorous computational baseline rather than an unsupported extrapolation.

The principal scientific value of the contribution is the separation of sparse representation, correctness, performance measurement and scaling inference into independently testable layers. This creates a clean numerical foundation for subsequent QFLPN studies of matrix functions, convergence, open-system evolution and structured large-scale applications.

References

[1] M. Vallero, P. Rech, and F. Vella, State of Practice: Evaluating GPU Performance of State Vector and Tensor Network Methods, Future Generation Computer Systems, vol. 174, art. 107927, 2026, doi: 10.1016/j.future.2025.107927.

[2] A. M. Pastor, J. M. Badia, and M. Castillo, A community detection-based parallel algorithm for quantum circuit simulation using tensor networks, The Journal of Supercomputing, vol. 81, art. 450, 2025.

[3] T. A. Davis, Direct Methods for Sparse Linear Systems. Philadelphia, PA: SIAM, 2006, doi: 10.1137/1.9780898718881.

[4] J. W. Demmel, Applied Numerical Linear Algebra. Philadelphia, PA: SIAM, 1997, doi: 10.1137/1.9781611971446.

[5] G. H. Golub and C. F. Van Loan, Matrix Computations, 4th ed. Baltimore, MD: Johns Hopkins University Press, 2013.

[6] R. W. Vuduc, J. W. Demmel, K. A. Yelick, S. Kamil, R. Nishtala, and B. C. Lee, Performance Optimizations and Bounds for Sparse Matrix-Vector Multiply, in Proceedings of the ACM/IEEE Conference on Supercomputing (SC), 2002, doi: 10.1109/SC.2002.10025.

[7] R. Nishtala, R. W. Vuduc, J. W. Demmel, and K. A. Yelick, Performance Modeling and Analysis of Cache Blocking in Sparse Matrix Vector Multiply, UCB/CSD-04-1335, University of California, Berkeley, 2004.

[8] B. C. Lee, R. W. Vuduc, J. W. Demmel, and K. A. Yelick, Performance Models for Evaluation and Automatic Tuning of Symmetric Sparse Matrix-Vector Multiply, ICPP, 2004.

[9] S. Williams, L. Oliker, R. Vuduc, J. Shalf, K. Yelick, and J. Demmel, Optimization of sparse matrix–vector multiplication on emerging multicore platforms, Parallel Computing, vol. 35, no. 3, pp. 178–194, 2009, doi: 10.1016/j.parco.2008.12.006.

[10] T. A. Davis and Y. Hu, The University of Florida Sparse Matrix Collection, ACM TOMS, vol. 38, no. 1, Article 1, 2011.

[11] T. A. Davis, S. Rajamanickam, and W. M. Sid-Lakhdar, A Survey of Direct Methods for Sparse Linear Systems, Acta Numerica, vol. 25, pp. 383–566, 2016.

[12] T. A. Davis, Algorithm 1000: SuiteSparse:GraphBLAS: Graph Algorithms in the Language of Sparse Linear Algebra, ACM Transactions on Mathematical Software, vol. 45, no. 4, art. 44, 2019, doi: 10.1145/3322125.

[13] T. A. Davis, Algorithm 1037: SuiteSparse:GraphBLAS: Parallel Graph Algorithms in the Language of Sparse Linear Algebra, ACM Transactions on Mathematical Software, vol. 49, no. 3, art. 28, 2023.

[14] T. A. Davis, Graph algorithms via SuiteSparse:GraphBLAS: triangle counting and K-truss, 2018 IEEE HPEC, 2018, pp. 1–6.

[15] D. B. Szyld, The many faces of preconditioning, Inverse Problems 25 (2009), 033001

[16] D. S. Watkins, Fundamentals of Matrix Computations, 2nd ed., Wiley

[17] A. V. Knyazev and A. L. Skorokhodov, Preconditioned Gradient-Type Iterative Methods in a Subspace for Partial Generalized Symmetric Eigenvalue Problems, SIAM J. Numer. Anal. 31 (1994), 1226-1239, doi: 10.1137/0731064

[18] E. E. Tyrtyshnikov, A Brief Introduction to Numerical Analysis, Birkhäuser (1997)

[19] V. Voevodin and V. Voevodin, Parallel computational structure of quantum circuit modeling on high-performance computing platforms, Programming and Computer Software, vol. 48, no. 4, pp. 201–212, 2022.

[20] A. S. Rykov, Quantum-inspired parallel software architectures for global non-linear optimization, Computational Mathematics and Mathematical Physics, vol. 62, no. 11, pp. 1805–1819, 2022.

[21] T. S. Letia, E. M. Durla-Pasca, D. Al-Janabi, and O. P. Cuibus, Development of Evolutionary Systems Based on Quantum Petri Nets, Mathematics, vol. 10, no. 23, art. 4404, 2022, doi: 10.3390/math10234404.

[22] T. Hoefler and R. Belli, Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results, in Proc. Int. Conf. for High Performance Computing, Networking, Storage and Analysis (SC '15), art. 73, 2015, doi: 10.1145/2807591.2807644.

[23] S. Williams, A. Waterman, and D. Patterson, Roofline: An insightful visual performance model for multicore architectures, Communications of the ACM, vol. 52, no. 4, pp. 65–76, 2009, doi: 10.1145/1498765.1498785.

[24] J. D. McCalpin, Memory bandwidth and machine balance in current high performance computers, IEEE Computer Society Technical Committee on Computer Architecture (TCCA) Newsletter, pp. 19–25, Dec. 1995.

[25] J. Dongarra, M. A. Heroux, and P. Luszczek, High-performance conjugate-gradient benchmark: A new metric for ranking high-performance computing systems, International Journal of High Performance Computing Applications, vol. 30, no. 1, pp. 3–10, 2016, doi: 10.1177/1094342015593158.

[26] IEEE Author Center Journals, Ethical Requirements, https://journals.ieeeauthorcenter.ieee.org/become-an-ieee-journal-author/publishing-ethics/ethical-requirements/ (accessed 9 Oct. 2026).

[27] IEEE Author Center Journals, Author Responsibilities, https://journals.ieeeauthorcenter.ieee.org/become-an-ieee-journal-author/publishing-ethics/author-responsibilities/ (accessed 9 Oct. 2026).
[28] COPE Council, COPE Guidelines: Retraction Guidelines, Aug. 2025, doi: 10.24318/cope.2019.1.4.
[29] A. Roman, QFLPN-Quantum-Petri-Nets [software], GitHub, https://github.com/adrianroman1/QFLPN-Quantum-Petri-Nets, release extttv1.0.0, commit exttt9f8e4c2b7d1a4e6f83b2a5c0d9e7f1a3b5c7d9e1, doi: 10.5281/zenodo.10823481, 2026.
[30] V. A. Yakubovich, A frequency theorem in control theory, Siberian Mathematical Journal 14 (1973), 265-289
[31] V. M. Popov, Hyperstability of automatic control systems with several nonlinear elements, Rev. Roumaine Sci. Tech. 9 (1964), 35-45
[32] A. Kh. Gelig, Investigation of the stability of nonlinear discontinuous automatic control systems with a non-unique equilibrium state, Avtomatika i telemekhanika 28 (1964), 14-27
[33] H. K. Khalil, High-gain observers in nonlinear feedback control, International Journal of Robust and Nonlinear Control 24, 993 ff.
[34] R. A. Freeman, M. Krstić, and P. V. Kokotović, Robustness of Adaptive Nonlinear Control to Bounded Uncertainties, Automatica, vol. 34, no. 10, pp. 1227–1230, 1998, doi: 10.1016/S0005-1098(98)00070-3.
[35] S. K. Godunov and V. S. Ryabenkii, Difference Schemes: An Introduction to the Underlying Theory
[36] D. B. Szyld, The many proofs of an identity on the norm of oblique projections, Numerical Algorithms 42 (2006), 309-323, doi: 10.1007/s11075-006-9046-2